Busfahrt

Nachdem die Trainer entschieden haben, dass das Wiencamp dieses Jahr in einer Almhütte in Tirol abgehalten werden soll, hätten die Teilnehmer:innen den Trainern die günstigste Zugverbindung berechnen sollen. Leider hat es aber niemand geschafft das Problem effizient zu lösen und somit haben die Trainer weiterhin ein Problem.

Doch anstatt eine bessere Zugverbindung zu suchen, hat Nik einen genialen Vorschlag: Da die Ölpreise in letzter Zeit so stark gesunken sind, ist es günstiger für das Trainingslager nach Island zu fliegen! Dort können sie dann einen gelenkigen Bus mieten und zu einer Nicht-ganz-Alm-Hütte in Thorsmörk fahren und endlich das Trainingscamp abhalten.

Die anderen Trainer sind begeistert und der Vorschlag ist sofort angenommen. Jetzt muss nur noch eine passende Hütte gefunden werden. Doch wieder läuft nicht alles wie geplant. Island ist viel windiger als Österreich und aufgrund der großen Angriffsfläche des Busses kann dieser auf einer Straße nur dann sicher fahren, wenn die aktuelle Windstärke k nicht größer ist als eine bestimmte Windstärke w, die für jede Straße individuell festgelegt wurde. Eine Straße verbindet immer zwei Orte u und v. Es ist garantiert, dass es einen Weg von jedem Ort zu jedem anderen Ort gibt, wenn kein Wind weht.

Da Wolfgang keine Lust auf den Papierkram, den ein umgekippter Bus mit sich bringen würde, hat, haben die Trainer beschlossen, sich an die Empfehlung zu halten.

Damit die Planung dieses Mal erfolgreich ist und das Trainingscamp zustande kommt, möchten die Trainer nach der durchwachsenen letzten Erfahrung die Planung diesmal nicht in die Hände der Teilnehmer:innen legen, sondern selbst entscheiden. Doch leider müssen sie nicht nur die An- und Abreise planen, sondern auch Tagesausflüge von verschiedensten Orten zu anderen Orten. Damit sie schnell entscheiden können, ob sie von einem Ort zu einem anderen Ort kommen können, wenn eine bestimmte Windgeschwindigkeit vorherrscht, wünschen sie sich ein Tool, das genau das berechnen kann. Außerdem ist die Karte von Island zurzeit noch unvollständig und die Trainer wissen bereits, dass sie während der Recherche laufend neue Straßen entdecken werden. Diese sollen natürlich in zukünftigen Anfragen berücksichtigt werden. Maximal muss das Tool q Anfragen bearbeiten können, jedoch ist noch nicht klar, wie die Anfragen auf "Insert" und "Query" Anfragen verteilt sind. Natürlich sind die Trainer dazu in der Lage, ein solches Tool zu programmieren, jedoch sind sie zu faul und möchten den Teilnehmer:innen noch eine zweite Chance geben.

Zeig den Trainern, dass sie ihr Vertrauen zurecht in die Teilnehmer:innen gesetzt haben, indem du ein Programm dafür schreibst.

(Anmerkung: Wenn du die Trainer für ihre Faulheit bestrafen möchtest und *keine* Lösung programmierst, gibt es auch kein Trainingscamp und die Trainer fahren alleine!)

Implementierung

Implementiere die Funktionen

- `void init(int n, int q, vector<vector<int>> edges)` Wobei `n` die Anzahl an Knoten, `q` die Anzahl der Anfragen und `edges` ein 2d-Vektor der Länge `m` ist und jedes Element selbst 3 Elemente beinhaltet: `edges[i][0]` und `edges[i][1]` sind die Knoten, die durch die Kante verbunden werden, und `edges[i][2]` ist die maximale Windstärke,

die die Straße, die diese Kante darstellt, aushält.

- `bool query(int u, int v, int k)`, wobei `u` und `v` Start und Endknoten sind und `k` die Windstärke ist, die aktuell vorherrscht. Die Funktion soll `true` zurückgeben, wenn es einen Pfad von `u` nach `v` gibt, der nur Straßen benutzt, die eine maximale Windstärke von mindestens `k` haben, und `false` zurückgeben, wenn es keinen solchen Pfad gibt.
- `void insert(int u, int v, int w)`, wobei `u` und `v` die Knoten sind, die durch die neue Kante (= Straße) verbunden werden, und `w` die maximale Windstärke ist, die die neue Straße aushält.

Beispiele

Eingabe	Ausgabe
5 8 5 1 2 5 1 5 6 1 4 9 2 5 15 2 3 9 2 4 7 3 5 7 4 5 2 q 4 3 4 q 5 4 7 q 3 3 30 q 2 1 7 q 1 3 8	1 1 1 1 0

Eingabe	Ausgabe
3 2 6 1 2 3 2 3 4 q 1 3 2 q 3 2 5 q 2 1 4 i 1 3 10 q 1 3 8 q 2 1 4	1 0 0 1 1

Subtasks

Allgemein gilt:

- $1 \leq n, q \leq 10^5$
- $n - 1 \leq m \leq 5 \cdot 10^5$
- $1 \leq \textit{Maximale Windstärke} \leq 10^9$
- $1 \leq u, v \leq n$
- $1 \leq k \leq 10^9$

Subtask 1 (12 Punkte): Es gibt keine Insert-Anfragen, $1 \leq n \leq 100$, $1 \leq q, m \leq 1000$

Subtask 2 (16 Punkte): Es gibt keine Insert-Anfragen, $1 \leq n \leq 700$, $1 \leq q \leq 1000$, $m \leq 2.5 \cdot 10^5$, die Anfragen sind in aufsteigender Reihenfolge der Windstärke sortiert

Subtask 3 (27 Punkte): Es gibt keine Insert-Anfragen, die Anfragen sind in aufsteigender Reihenfolge der Windstärke sortiert

Subtask 4 (28 Punkte): Es gibt keine Insert-Anfragen

Subtask 5 (7 Punkte): $1 \leq n \leq 100$, $1 \leq q, m \leq 1000$

Subtask 6 (10 Punkte): Keine Einschränkungen

Limits

Zeitlimit: 2 s **Speicherlimit:** 512 MB

Eingabe

Das Einlesen der Eingabe wird von einem von uns geschriebenen Programm, dem Grader, übernommen. In der ZIP-Datei zu dieser Aufgabe findest du eine grader-Datei, die du zum Testen deiner Lösung mit deinem Programm kompilieren kannst.

Dieser Grader liest die Eingabe im folgenden Format ein: Die erste Zeile enthält drei Zahlen n, m und q, die Anzahl der Knoten, Kanten und Anfragen. Die nächsten m Zeilen enthalten jeweils drei Zahlen u, v und w, die eine ungerichtete Kante zwischen den Knoten u und v mit maximaler Windstärke w beschreiben. Die nächsten q Zeilen enthalten jeweils einen Character t und drei Zahlen u, v und w/k. t gibt an, ob es sich um eine Query ('q') oder eine Insert-Anfrage ('i') handelt. u, v und w/k sind die Parameter der jeweiligen Anfrage, wie sie in der Implementierung beschrieben wurden.

Eingabe	**Ausgabe**
$n\ \ m\ \ q$ $u_1\ \ v_1\ \ w_1$ $\vdots$ $u_m\ \ v_m\ \ w_m$ $t_1\ \ u_1\ \ v_1\ \ w_1/k_1$ $\vdots$ $t_q\ \ u_q\ \ v_q\ \ w_q/k_q$	$result$